

Assignment Brief

Faculty of Computing and Information Technology

Diploma in Computing and Information Technology

This sheet to be completed for each assignment/assessment/project brief

Submission Deadlines		Marks and Feedback
Before 3.00 p.m. on:		15 working days after deadline (L5)
Midterm Assignment 2	19 June 2026	
Date of issue:	5 June 2026	

Level	4	Unit code	CS 2108
Unit title	Object Oriented Programming		
Assignment number & title	Mid-term Assessment 2		
Assessment type	Individual Assignment		
Weighting of assessment	20%		
Unit learning outcomes	1. Demonstrate the following knowledge and understanding 1.2 Produce a set of use cases for a problem statement 2. Demonstrate the following skills and abilities 2.1. Proficient in writing object-oriented programs in Java.		

Completing your assignment
What am I required to do in this assignment?

You are required to write two GUI applications by using OOP features.

1. Overview

This assessment tests your ability to utilize program flow controls (loops, conditional statements) and logical operators in Java to solve advanced programming problems using the **JavaFX** graphical framework. You are required to develop a suite of tools for a **"Smart City Management Dashboard."**

2. Learning Outcomes

This assessment contributes to the following Learning Outcomes:

- **LO 1.2:** Produce a set of use cases for a problem statement.
- **LO 2.1:** Proficient in writing object-oriented programs in Java.

3. Tasks & Requirements

Task 1: Membership Tier Evaluator (Conditionals & JavaFX)

Scenario: The Smart City rewards citizens based on their annual contribution/spending.

- **Task:** Design a JavaFX interface where users enter an annual purchase amount and select a membership tier (Bronze, Silver, or Gold).
- **Logic Requirements:** Implement a calculate () method that uses switch or if-else statements to determine:
 - **Bronze:** 5% discount, 1 point per \$10 spent.
 - **Silver:** 10% discount, 2 points per \$10 spent.
 - **Gold:** 15% discount, 5 points per \$10 spent.
- **JavaFX Elements:** Use TextField for inputs, RadioButton for tiers, and a Button to trigger the logic. Display results in a Label.

Task 2: Performance Analyzer (Loops & JavaFX)

Scenario: Analyze the efficiency scores of various city sectors (e.g., Power, Water, Transport).

- **Task:** Develop a JavaFX application to process a list of efficiency scores.
- **Logic Requirements:** Use a for or while loop to iterate through an ArrayList<Double> of scores entered by the user.
 - Calculate the **average efficiency score**.
 - Identify the **highest score** using relational operators.
- **JavaFX Elements:** Use a TextField to add scores, a Button to process them, and a TextArea to display the list of scores and the final analysis.

Task 3: Secure Access Validator (Complex Logic & JavaFX)

Scenario: Citizens must set up a secure PIN/Password for the City Vault.

- **Task:** Create a secure password verification interface.
- **Logic Requirements:** Use logical operators (&&, ||, !) to validate that a password meets the following "Cyber-Security" criteria:
 - Minimum **8 characters**.
 - At least **one digit**.
 - At least **one special character** (e.g., !, @, #, \$, %).
- **JavaFX Elements:** Use a PasswordField for input and a Button to validate. Display validation results using a JavaFX Alert dialog or a dynamic Label.

4. Submission Requirements

- 1 **Technical Report:** A document containing:
 - **Use Case Table:** A set of use cases for the three problem statements above (LO 1.2).
 - **Screenshots:** Evidence of the working GUI for each task.
- 2 **Source Code:** Three Java files (one for each task) following professional naming conventions.

5. Assessment Load

- **Preparation:** 20 Hours
- **Completion:** 3 Hours
- **Total:** 23 Hours

What do I need to do to pass?

(Contributory evidence for Threshold expectations)

Task / Question Number	Marks	Threshold Expectations
Task 1: Membership Tier Evaluator	20 Marks	Implement a functional JavaFX GUI for membership tier evaluation, correctly applying conditional logic for discounts and points, with basic input validation.
Task 2: Performance Analyzer	25 Marks	Implement a functional JavaFX GUI for score analysis, correctly using loops and ArrayLists to calculate average and highest scores, with basic input validation.
Task 3: Secure Access Validator	25 Marks	Implement a functional JavaFX GUI for password validation, correctly applying logical operators for complexity rules, with basic input validation.
Mid-Term A2 (20%)	70 Marks (Practical)	Implement Java GUI applications by using OOP features (Learning Outcomes 1, 2).
Technical Report	30 Marks (Documentation)	Produce a set of use cases for the problem statements (LO 1.2) and provide screenshots of working GUIs.

Submission Requirements

1. Technical Report: A document containing:
 - Use Case Table: A set of use cases for the three problem statements above (LO 1.2).
 - Screenshots: Evidence of the working GUI for each task.
2. Source Code: Three Java files (one for each task) following professional naming conventions.

How will my assignment be marked?

Your assignment will be marked according to the threshold expectations.

Part 1: Documentation & Design (30 Marks)

Criteria	Excellent (80%+)	Good (60-79%)	Satisfactory (40-59%)	Needs Improvement (0-39%)
Use Case Analysis (LO 1.2) (15 Marks)	Comprehensive use case table for all 3 tasks with clear actors, triggers, and flows.	Clear use case table, but may miss minor edge case flows.	Basic use case table; lacks detail in some steps.	Incomplete or missing use case analysis.
GUI Design & Layout (15 Marks)	Professional JavaFX layout using appropriate panes (HBox, VBox, Grid). Excellent user feedback.	Functional layout, mostly organized. Good use of UI elements.	Basic layout; UI elements are cluttered or unaligned.	Poor layout; UI elements are missing or non-functional.

Part 2: Practical Implementation (70 Marks)

Task 1: Membership Tier Evaluator (20 Marks)

Criteria	Excellent (80%+)	Good (60-79%)	Satisfactory (40-59%)	Needs Improvement (0-39%)
Function Implementation (8 Marks)	Three or more well-implemented user-defined methods (e.g., <u>calculateBenefits</u> , <u>calculateDiscount</u> , <u>calculateLoyaltyPoints</u>). Clear, distinct, and modular logic.	Program includes user-defined methods, but may lack modularity or clear separation of concerns.	Functions implemented, but some may not perform distinct tasks. Logic may be primarily within event handlers.	Little to no modularity; lacks distinct functions for calculation logic.
Input Handling & Validation (6 Marks)	Excellent input handling: <u>TextField</u> for amount, <u>RadioButton</u> for tiers. Robust validation for non-numeric/negative input.	Good input handling, but may have minor issues with validation or error messages (e.g., generic "Error").	Basic input handling; some errors caught, but not comprehensive (e.g., negative numbers not handled).	Little or no input validation, leading to crashes or poor user experience.
Functionality & Correctness (6 Marks)	The application runs perfectly with correct outputs for all tested cases, including various amounts and tiers.	Runs as expected, but minor issues in handling certain tier selections or edge cases.	Runs, but some calculations produce incorrect results or it may not handle all inputs as expected.	Multiple errors or crashes, incorrect outputs, or key functionality missing.

Task 2: Performance Analyzer (25 Marks)

Criteria	Excellent (80%+)	Good (60-79%)	Satisfactory (40-59%)	Needs Improvement (0-39%)
Loops & Data Structures (15 Marks)	Correct and efficient use of <u>for</u> or <u>while</u> loops to iterate through <u>ArrayList<Double></u> . Accurate calculation of average, highest, and lowest scores.	Correct use of loops and <u>ArrayList</u> , but may have minor inefficiencies or slight inaccuracies in calculations.	Basic use of loops and <u>ArrayList</u> ; calculations may be partially correct or lack robustness.	Incorrect use of loops or <u>ArrayList</u> ; calculations are flawed or missing.
Input Handling & Validation (5 Marks)	Robust input validation for numeric scores (e.g., 0-100 range). Clear error messages for invalid inputs.	Good input validation, but may miss minor edge cases or have generic error messages.	Basic input validation; some invalid inputs may not be handled.	Little to no input validation, leading to crashes or poor user experience.

Criteria	Excellent (80%+)	Good (60-79%)	Satisfactory (40-59%)	Needs Improvement (0-39%)
UI Interaction & Display (5 Marks)	Effective use of <u>TextField</u> for adding scores and <u>TextArea</u> for displaying a clear list of scores and analysis results.	Functional UI interaction; display may be slightly cluttered or lack some clarity.	Basic UI interaction; display is functional but not well-organized.	Poor UI interaction; display is confusing or non-functional.

Task 3: Secure Access Validator (25 Marks)

Criteria	Excellent (80%+)	Good (60-79%)	Satisfactory (40-59%)	Needs Improvement (0-39%)
Logical Operators & Complexity Rules (15 Marks)	Proficient use of <u>&&</u> , `		<u>,</u> <u>and!</u> ` to enforce all three complexity rules (length, digit, special character). Logic is robust and handles all cases.	Correct use of logical operators, but may have minor flaws in enforcing one or more complexity rules.
UI Elements & Feedback (10 Marks)	Professional use of <u>PasswordField</u> for input and JavaFX <u>Alert</u> dialogs or dynamic <u>Label</u> for clear, specific validation results.	Good use of <u>PasswordField</u> and feedback mechanism, but messages may be generic or less intuitive.	Basic use of <u>PasswordField</u> ; feedback is minimal or unclear.	Poor UI implementation; password field or feedback mechanism is non-functional or missing.

Grading Standards

- **Excellent:** Code is modular, well-commented, and handles all edge cases (e.g., non-numeric input in Task 1/2).
- **Good:** Functional code with minor naming or commenting issues.
- **Satisfactory:** Code works but lacks error handling or is not modular.
- **Needs Improvement:** Code does not run or logic is fundamentally flawed.

Student Declaration	
I certify that the work submitted for this assignment is my own. I have clearly referenced any sources used in the work (e.g. books, websites, journals). I confirm that I have read, understood, and agreed to abide by the IT, IP, GitHub usage policies of STI and I understand that any false declaration is a form of malpractice.	
Student Signature:	Date: